Summary of the Document:

The document outlines two main problems for a take-home assignment in Applied AI/ML Engineering, focusing on building and evaluating RAG (Retrieval-Augmented Generation) systems and creating a robust evaluation pipeline for LLM outputs.

Problem 1: Cost-Efficient RAG Application

Objective:

Develop a QA service over a document corpus using a low-cost vector store, with a focus on cost-efficiency, retrieval quality, answer quality, and latency. The system should be capable of ingesting and processing documents (PDF, HTML, MD), embedding chunks, and retrieving relevant information to generate grounded answers.

Key Requirements:

1. **Ingestion:** Support for PDF/HTML/MD formats with configurable chunk size and overlap. Ensure idempotent re-ingestion to avoid duplicate vectors.
2. **Embedding & Storage:** Embed chunks using a specified model and store vectors along with metadata, including at least one metadata filter.
3. **Retrieval:** Implement top-k retrieval with a parameter k, and generate answers that cite the relevant chunks. Handle cases where no relevant context is found without hallucination.
4. **Exposure:** Provide the service via HTTP endpoint or CLI, with configurable settings via environment variables. Log per-query metrics such as latency, chunk count, and token usage.

Evaluation Metrics:

- **Retrieval:** Recall@k, Hit Rate, MRR, nDCG@k, and context precision.
- **Answer Quality:** Faithfulness, relevance, and optionally EM/F1 if gold answers are available.
- **Cost & Latency:** Compare monthly costs for 100K, 1M, and 10M vectors against a managed DB. Report p50/p95 retrieval latency and trade-offs accepted.

Deliverables:

- A **runnable RAG service**.
- An **evaluation harness** and results file (including all three layers: retrieval, answer, and cost).
- A **cost-comparison table**.
- A **README** with a discussion on when to switch back to a managed DB and whether retrieval or generation was the weak link.

Scoring (100 points):

- **Correctness & Ingestion (20 pts):** End-to-end functionality, idempotency, and chunking.
- **Retrieval Evaluation (20 pts):** Accurate computation of IR metrics.
- **Answer Evaluation (20 pts):** Faithfulness and relevance, not just assertion.
- **Cost Analysis (20 pts):** Credible comparison with assumptions.
- **Engineering & Clarity (20 pts):** Structure, configuration hygiene, README, and git history.

Problem 2: LLM-as-Judge Evaluation Pipeline

Objective:

Build a pipeline that uses an LLM as a judge to evaluate the quality of model outputs, while addressing and mitigating judge biases.

Key Requirements:

1. **Judging Modes:** Implement at least one of the following: pointwise scoring, pairwise A-vs-B, reference-based, or reference-free.
2. **Test Suite Input:** Accept test cases in JSON/YAML format, and generate structured prompts for the judge.
3. **Rubric Definition:** Define an explicit rubric (e.g., correctness, faithfulness, completeness, instruction-following, tone, safety) rather than just numerical scores.
4. **Aggregation & Reporting:** Generate per-case and suite-level reports (pass rate, mean scores, win rate), and support comparisons between two configurations (e.g., prompt v1 vs v2 or model A vs B).
5. **Logging & Auditing:** Log all judge prompts and raw responses for replayability. Track tokens and calls for the judge and generator. Allow independent configuration of the judge and generator.

Bias Handling:

- **Position (A/B Order):** Run each pair in both orders and average or require agreement. Report the flip rate.
- **Verbosity / Length:** Control for answer length by using padded answers and penalizing unsupported lengths.

Conclusion:

The document presents two distinct but complementary challenges:

1. **Building a cost-efficient RAG system** with rigorous evaluation of retrieval, answer quality, and cost.
2. **Creating a robust evaluation pipeline** for LLM outputs, with a focus on detecting and mitigating judge biases.

Both problems emphasize **engineering rigor**, **evaluation depth**, and **practical implementation** of AI/ML systems.